

Estruturas do Sistema Operacional

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Serviços do sistema operacional
- Interface do SO com usuário
- Chamadas ao sistema
- Programas de sistema
- Projeto e implementação de SOs
- Estrutura do SO
- Máquinas virtuais
- Depuração, geração e boot do SO

Serviços do sistema operacional

- Serviços providos pelo SO: para usuários e para eficiência do próprio SO
 - Variam muito de SO para SO: PCs, servidores, sistemas embarcados, sistemas móveis
 - Também visam tornar a programação mais simples: APIs

Serviços do sistema operacional

- Serviços que proveem funções úteis ao usuário:
 - Interface com usuário (UI)
 - Pode ser via linha de comando (CLI) ou gráfica (GUI)
 - Execução de programas: SO deve ser capaz de carregar programa na memória, executar e terminar
 - Realizar operações de E/S (ex.: mouse, teclado, disco)
 - Usuário acessa não dispositivos de E/S diretamente
 - Manipulação de sistemas de arquivos:
 - Ler/escrever arquivos, criar/apagar/buscar arquivos/diretórios: via programas de sistemas e API

Serviços do sistema operacional

- (cont.)

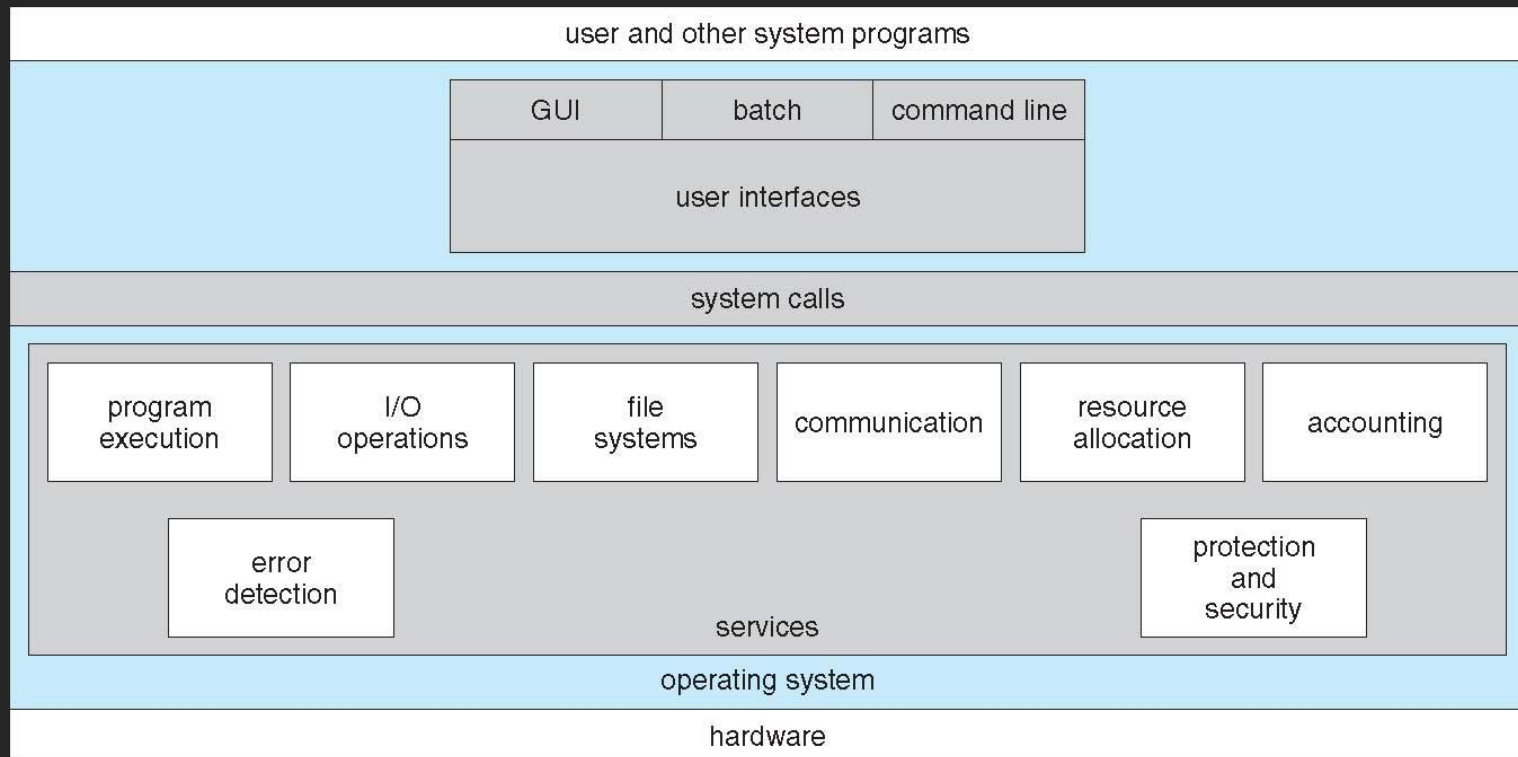
- Comunicação: processos necessitam trocar mensagens no mesmo computador ou externamente
 - Mecanismos: memória compartilhada, passagem de mensagens
- Detecção de erro em hardware/software
 - Ex.: dispositivos de E/S (erro de paridade, falha de conexão com impressora), em processos (divisão por zero, acesso ilegal à memória)
 - SO provê formas de contornar erros e meios de depuração

Serviços do sistema operacional

- Serviços para garantir eficiência do sistema:
 - Alocação de recursos: múltiplos usuários e processos concorrendo pela CPU, memória, dispositivos de E/S
 - Critérios para garantir eficiência, prioridade, justiça
 - Contabilidade: quais usuários/processos usam que quantidade de quais recursos?
 - Estatísticas para bom funcionamento do sistema e até cobrar usuários
- Proteção e segurança: controlar acesso aos recursos e defesa contra ataques
 - “Uma corrente é tão forte quanto seu elo mais fraco”

Serviços do sistema operacional

- Serviços do SO:



Interface do SO com usuário

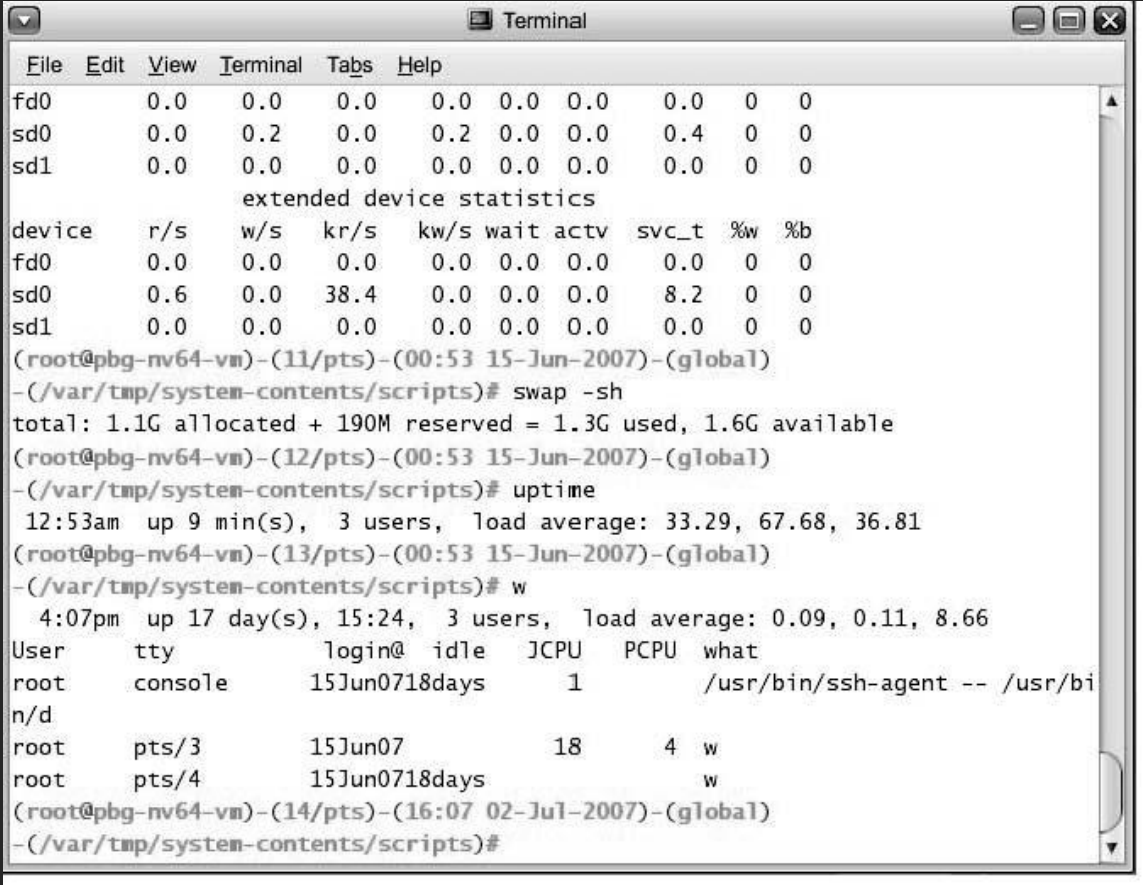
- Duas abordagens básicas: linha de comando (ou interpretador de comandos) e interface gráfica
- Interpretador de comandos: incluso no kernel por alguns SOs ou incluso como um programa especial (UNIX, Windows)
 - Utilidade: receber e executar próximo comando do usuário
 - Comandos: manipulação de arquivos/diretórios e execução de programas
 - Um SO pode ter diversos interpretadores de comandos (shells) com funcionalidades similares

Interface do SO com usuário

- Execução de comandos pode ser de duas formas:
 - Embutidos no interpretador de comandos: comando (ex.: deletar arquivo) é codificado no próprio interpretador
 - Complexidade do interpretador: quantidade/dificuldade dos comandos
 - Ex.: dir, copy, del do prompt de comando do Windows
 - Comandos não são implementados no interpretador
 - Interpretador apenas carrega programas para execução
 - Ex.: ls, cp, rm do Linux
 - cd é comando

Interface do SO com usuário

- Bourne Shell (sh):



```
File Edit View Terminal Tabs Help
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.0    0.2    0.0    0.2  0.0  0.0    0.4  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
      extended device statistics
device   r/s    w/s    kr/s    kw/s wait actv  svc_t  %w  %b
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.6    0.0   38.4    0.0  0.0  0.0    8.2  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
(root@pbg-nv64-vm)-(11/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# swap -sh
total: 1.1G allocated + 190M reserved = 1.3G used, 1.6G available
(root@pbg-nv64-vm)-(12/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# uptime
12:53am up 9 min(s), 3 users, load average: 33.29, 67.68, 36.81
(root@pbg-nv64-vm)-(13/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# w
4:07pm up 17 day(s), 15:24, 3 users, load average: 0.09, 0.11, 8.66
User      tty          login@ idle   JCPU   PCPU   what
root      console      15Jun0718days    1          /usr/bin/ssh-agent -- /usr/bi
n/d
root      pts/3        15Jun07          18     4    w
root      pts/4        15Jun0718days          w
(root@pbg-nv64-vm)-(14/pts)-(16:07 02-Jul-2007)-(global)
- (/var/tmp/system-contents/scripts)#
```

Interface do SO com usuário

- Interfaces gráficas: metáfora desktop (mesa de trabalho)
 - Intuitiva: user-friendly
 - Inventada na Xerox PARC: anos 70
 - Difundida pela Apple: anos 80
 - Windows 1.0: interface para o MS-DOS em 1985
 - Ícones representam arquivos, programas, ações, etc.
 - Baseada no uso do mouse
 - Diversas ações: mostrar informação/opções, executar programas, abrir diretórios (pasta/folder), alterar arquivos, etc.

Interface do SO com usuário

- GUI do MAC OS X:

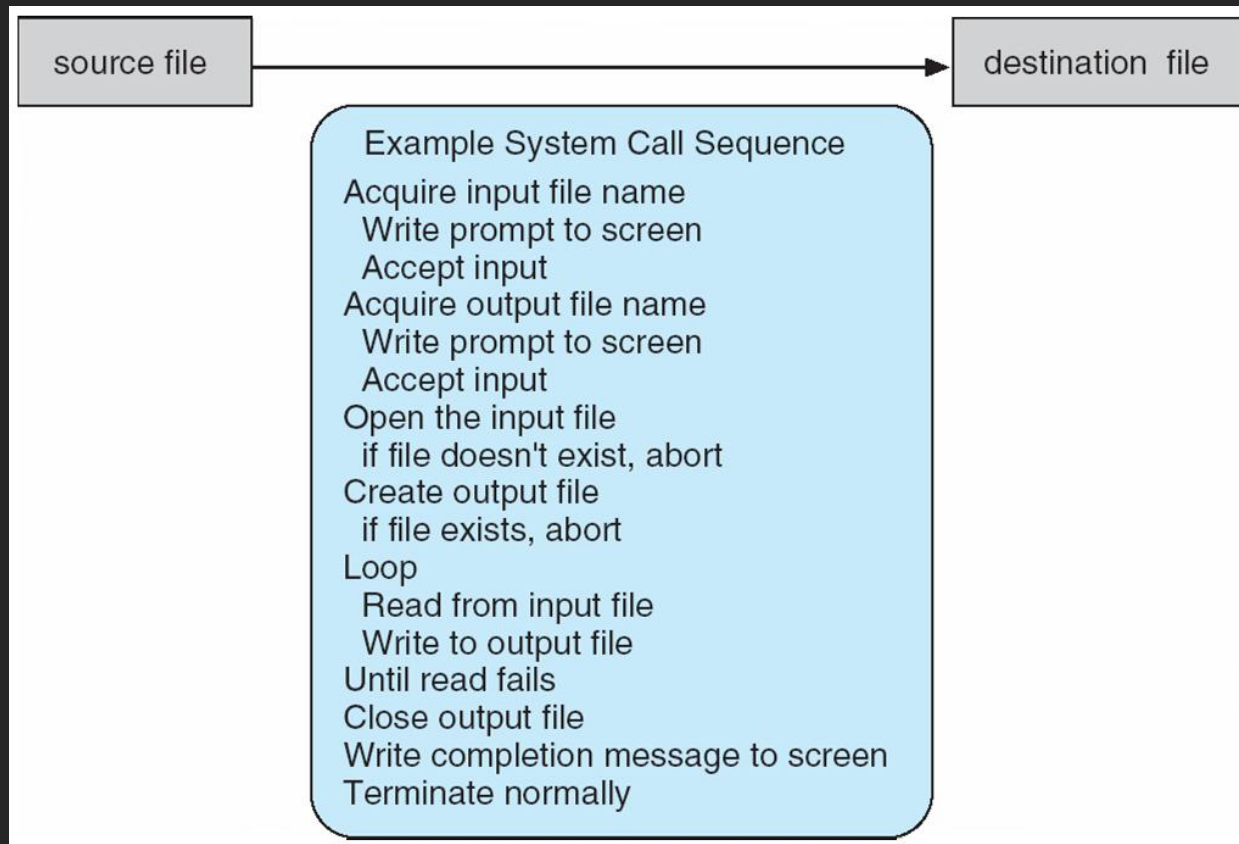


Chamadas ao sistema

- “As chamadas ao sistema (system calls) proveem uma interface aos serviços disponibilizados por um sistema operacional”
 - Interface de programação disponibilizada pelo SO
 - Geralmente implementadas em alto nível (C/C++)
 - Programas simples podem fazer milhares de chamadas ao SO
 - Invocação disparada por APIs de alto nível (ex.: `fopen()`)
 - Win32 API, POSIX API (UNIX, Linux e MAC OS X) e Java API

Chamadas ao sistema

- Chamadas ao sistema feitas para copiar arquivo:



Chamadas ao sistema

- Porque usar APIs ao invés de fazer a chamada ao SO?
 - Portabilidade: programa pode ser compilado em qualquer sistema que suporte API
 - Chamadas ao SO podem ser mais detalhadas e difíceis de lidar
 - Mas há forte correlação entre função da API e chamada ao SO correspondente

Chamadas ao sistema

- Exemplo de função de API no Linux: read()

```
#include <unistd.h>

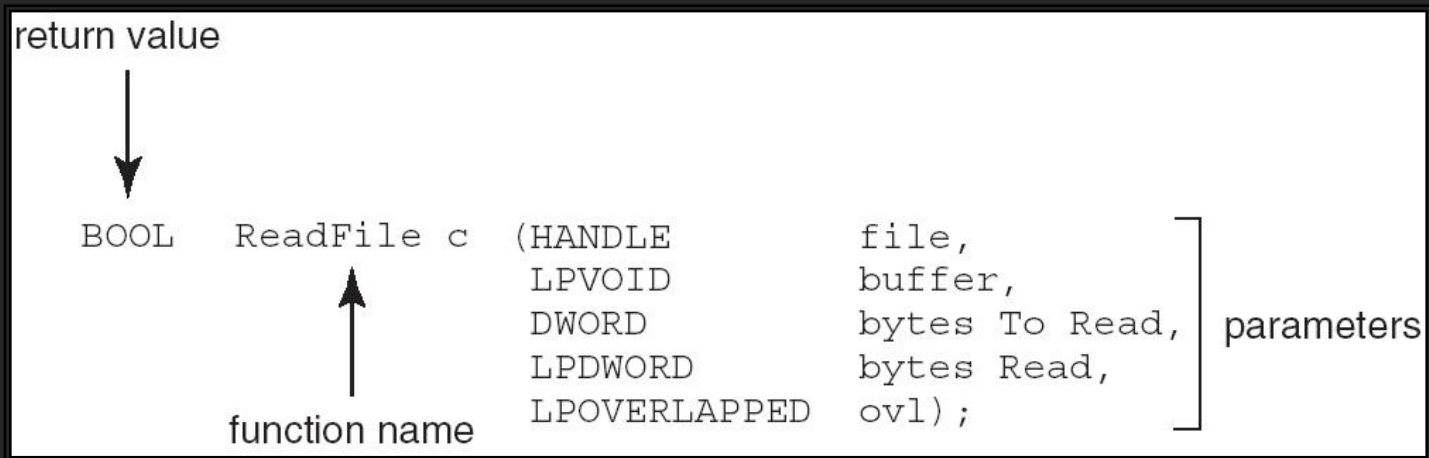
ssize_t read(int fd, void *buf, size_t count)
```

valor de retorno	nome da função	parâmetros
------------------	----------------	------------

- int fd: descritor de arquivo a ser lido
- void *buf: buffer para o qual os dados serão lidos
- size_t count: número máximo de bytes a serem lidos

Chamadas ao sistema

- Exemplo de função da API Win32: ReadFile()

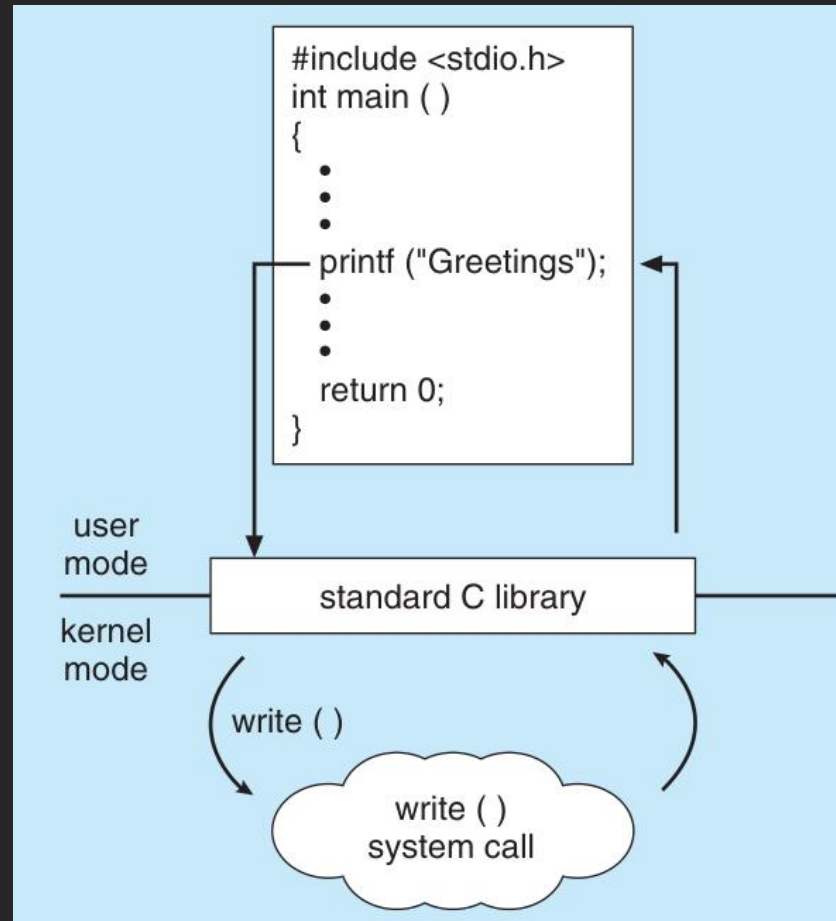


- `HANDLE file`: arquivo a ser lido
- `LPVOID buffer`: buffer para leitura
- `DWORD bytesToRead`: número de bytes a serem lidos e copiados para o buffer
- `LPDWORD bytesRead`: número de bytes lidos anteriormente
- `LPOVERLAPPED ovl`: indica se há sobreposição de operação de E/S

Chamadas ao sistema

- Implementação de chamadas ao SO:
 - Bibliotecas run-time (compilador) fornecem uma interface para as chamadas ao sistema
 - Ex.: `syscall()` declarada em `sys/syscall.h`
 - Um número é associado a cada chamada implementada
 - SO mantém uma tabela indexada por esses números
 - A interface invoca a chamada ao SO e retorna status/valores de retorno dessa chamada
 - Programador não precisa saber como chamada é implementada: apenas segue API do sistema

Chamadas ao sistema



Chamadas ao sistema

- “hello world” usando printf(), wrapper para chamada write() e chamada indireta usando syscall():

```
#include <stdio.h>
int main(void) {
    printf("hello, world!\n");
    return 0;
}
```

```
#include <unistd.h>
int main(void) {
    write(1, "hello, world!\n", 14);
    return 0;
}
```

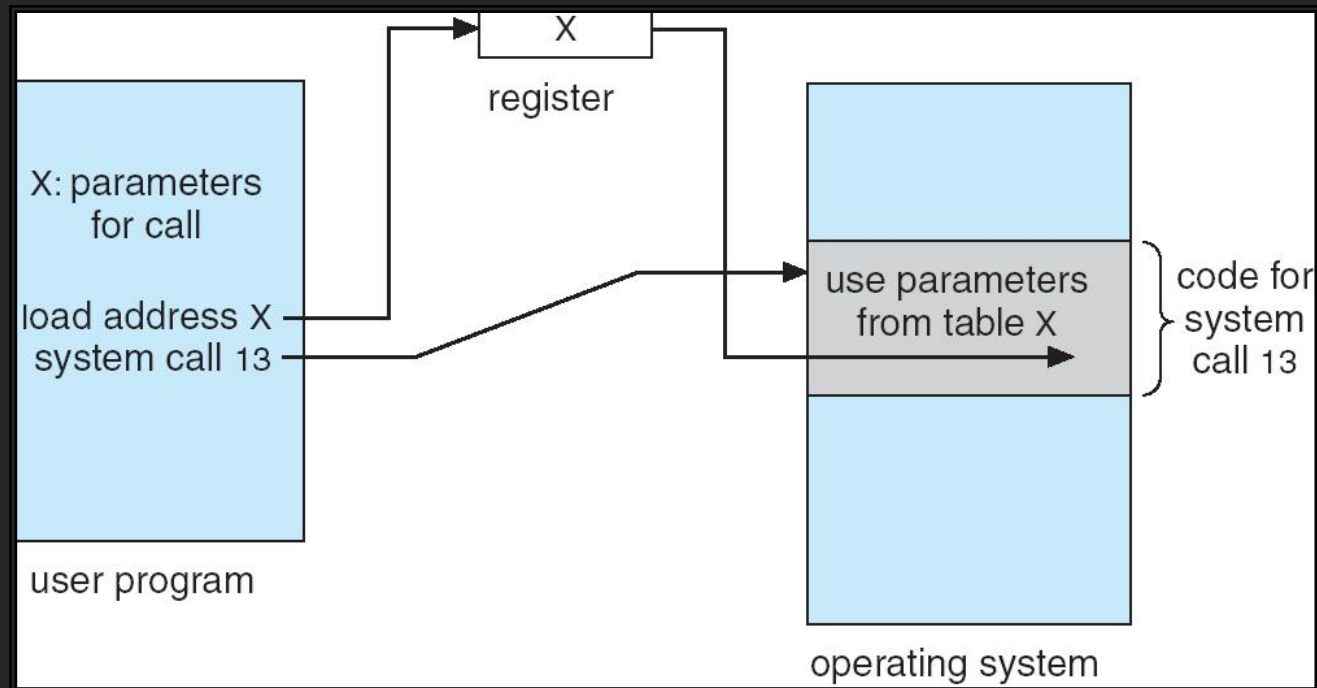
```
#include <unistd.h>
#include <sys/syscall.h>
int main(void) {
    syscall(SYS_write, 1, "hello, world!\n", 14);
    return 0;
}
```

Chamadas ao sistema

- Chamadas ao SO podem exigir zero ou mais parâmetros
 - Ex.: getpid, exit, open, write
- Passagem de parâmetros:
 - Por registradores: mais simples
 - Porém, limita quantidade/tamanho de parâmetros
 - Através de tabela na memória: registrador é usado para indicar endereço da tabela
 - Linux e Solaris
 - Empilhados antes da chamada

Chamadas ao sistema

- Passagem de parâmetros através de tabela:



Chamadas ao sistema

- Tipos de chamadas ao SO:
 - Gerenciamento de processos:
 - Criar, carregar, executar, terminar, abortar, ler/escrever atributos, esperar por tempo/evento, sinalizar evento, alocar e liberar memória
 - Gerenciamento de arquivos:
 - Criar e apagar arquivos/diretórios, abrir, fechar, ler, escrever, reposicionar, ler/escrever atributos
 - Gerenciamento de dispositivos:
 - Requisitar e liberar dispositivo, ler, escrever, reposicionar, ler/escrever atributos, conectar/desconectar logicamente dispositivos

Chamadas ao sistema

- (cont.):
 - Manutenção de informação:
 - Ler/alterar hora/data
 - Ler/escrever atributos de processos/arquivos/dispositivos
 - Comunicação:
 - Criar/terminar conexão
 - Enviar/receber mensagens
 - Ler informações de status de transferência
 - Conectar/desconectar dispositivos remotos
 - Proteção:
 - Ler/alterar permissões de arquivos/dispositivos, permitir/negar acesso a recursos

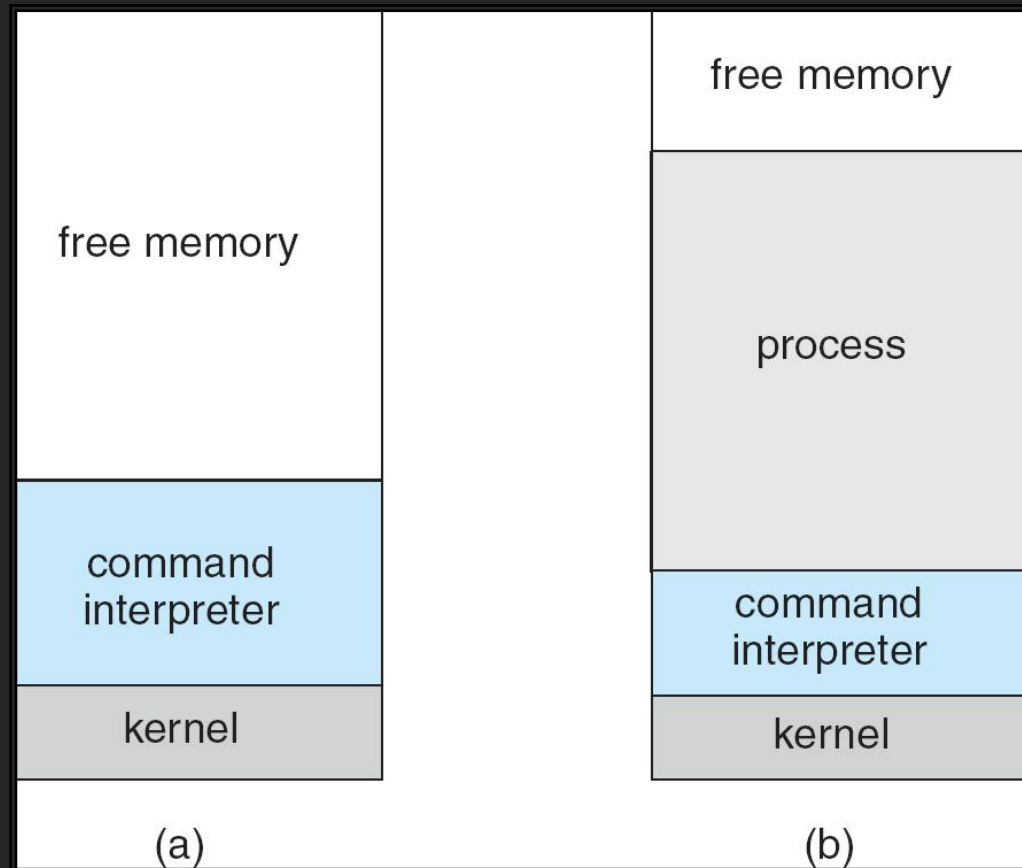
Chamadas ao sistema

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Chamadas ao sistema

- Execução de processos no MS-DOS (monotarefa):
 - A cada nova tarefa não necessita criar novo processo: apenas sobrescreve anterior
 - Parte do SO (interpretador de comandos) também é sobrescrita: mais memória disponível
 - Ao término da tarefa, o restante do interpretador é recarregado na memória

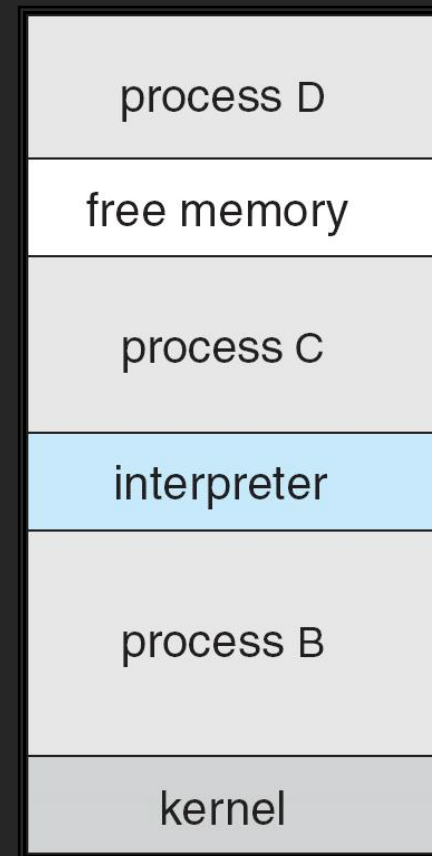
Chamadas ao sistema



(a) At system startup; (b) running a program

Chamadas ao sistema

- FreeBSD (multitarefa): necessita chamadas ao SO mais elaboradas
 - Shell realiza `fork()`
 - Programa é carregado: `exec()`



Programas de sistema

- “Programas de sistema (utilitários do sistema) proveem ambiente conveniente para execução/desenvolvimento de programas”
 - Tipos: manipulação de arquivos, informações do sistema, edição de arquivos, linguagem de programação, carregamento e execução de programas, comunicação
- Visão do SO por muitos usuários: definida pelos programas de sistema, não pelas chamadas ao SO

Programas de sistema

- Manipulação de arquivos:
 - Criar, apagar, copiar, renomear, imprimir, pesquisar, listar arquivos/diretórios
- Informações do sistema:
 - Saída formatada/impressão: data, hora, memória disponível, espaço em disco, número de usuários, versão do sistema, informações de log e depuração
- Edição de arquivos:
 - Editores de texto, programas para buscar/comparar/alterar arquivos

Programas de sistema

- Linguagem de programação:
 - Compiladores, montadores, depuradores, interpretadores
- Carregamento e execução de programas:
 - Carregadores absolutos/relocáveis, link editores, carregadores de overlays, depuradores
- Comunicação:
 - Envio/recebimento de mensagens para usuários, abrir páginas web, enviar/recebimento de e-mails, login remoto, transferência de arquivos

Projeto e implementação de SOs

- Projeto e implementação de SOs: algumas abordagens de sucesso
 - Estrutura interna de SOs podem variar muito
 - Inicialmente: definir metas e especificações
 - Tipo do hardware? Tipo do sistema?
 - Ex.: VxWorks (SO de tempo real para sistemas embarcados) e MVS (largamente multiusuário/multiacesso para mainframes)
 - Metas de usuário x metas do sistema
 - Metas de usuário: SO deve ser conveniente para usar, fácil de aprender, confiável, seguro, rápido
 - Metas do sistema: SO deve ser fácil de projetar/implementar/manter, flexível, confiável, livre de erros, eficiente

Projeto e implementação de SOs

- Princípios importantes a serem separados
 - Política: o que será feito?
 - Ex.: justiça entre usuários competindo pela CPU
 - Mecanismo: como será feito?
 - Ex.: implementação do algoritmo de escalonamento Round-Robin
- A separação de política/mecanismo permite flexibilidade
 - Mecanismos podem ser melhorados para atender política
 - Alteração na política leva a alteração nos mecanismos

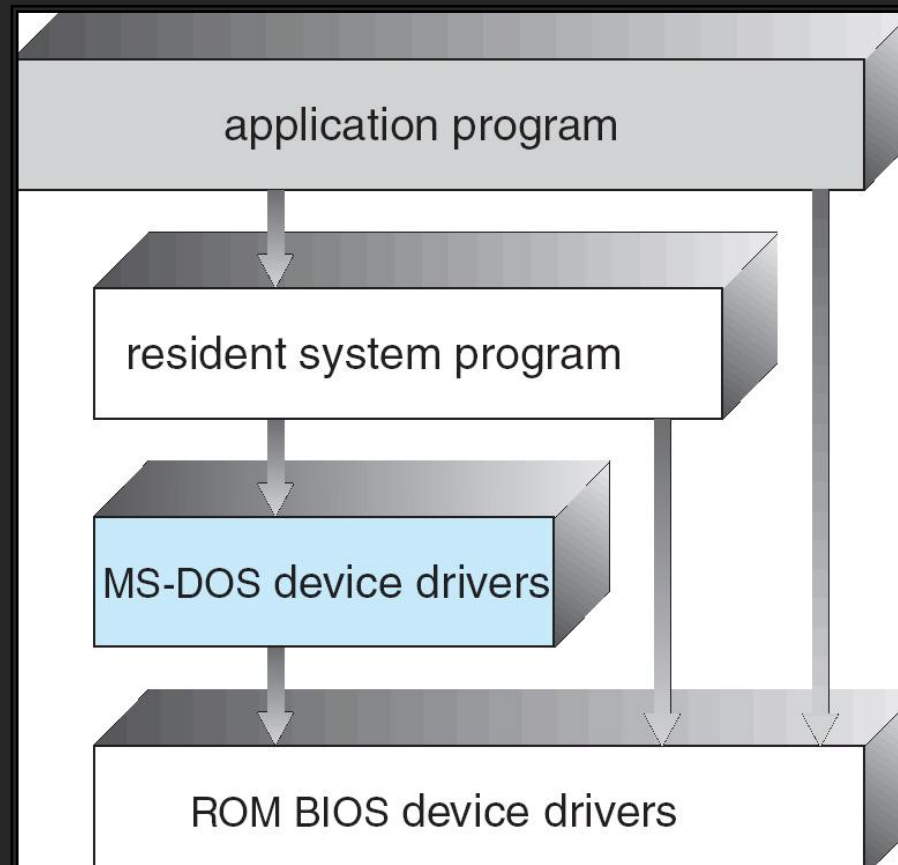
Estrutura do SO

- SO é um sistema grande e complexo
 - Projeto deve ser cuidadoso: funcionar e ser modificado facilmente
 - Abordagem comum: componentes/módulos bem definidos, ao invés de sistema monolítico
- Estrutura do SO: define como componentes são separados e interconectados

Estrutura do SO

- Estrutura simples: MS-DOS
 - Pequeno, simples, limitado
 - Hardware Intel 8088: sem modo dual
 - Meta: ocupar pouco espaço
 - Sem módulos bem definidos
 - Programas de aplicação podem acessar diretamente dispositivos de E/S
 - Liberdade para escrever aplicações mais rápidas
 - Programas com erros/maliciosos afetam todo sistema

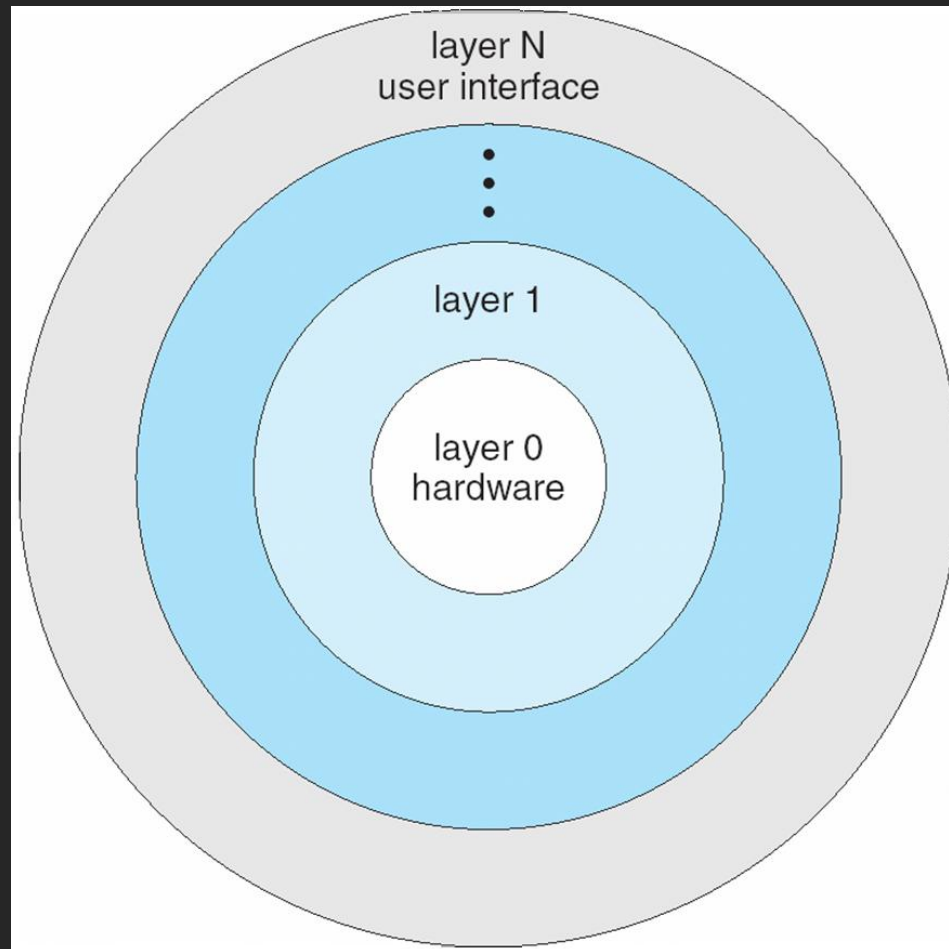
Estrutura do SO



Estrutura do SO

- Abordagem em camadas:
 - SO dividido em diversas camadas/níveis
 - Camada 0: hardware
 - Camada N: interface com usuário
 - Camada utiliza funcionalidades das inferiores e disponibiliza outras funcionalidades para as superiores
 - Vantagem: implementação mais simples
 - Mais fácil construir e depurar
 - Facilita também a manutenção
 - Dificuldades:
 - Definir camadas, overhead adicionado por camada

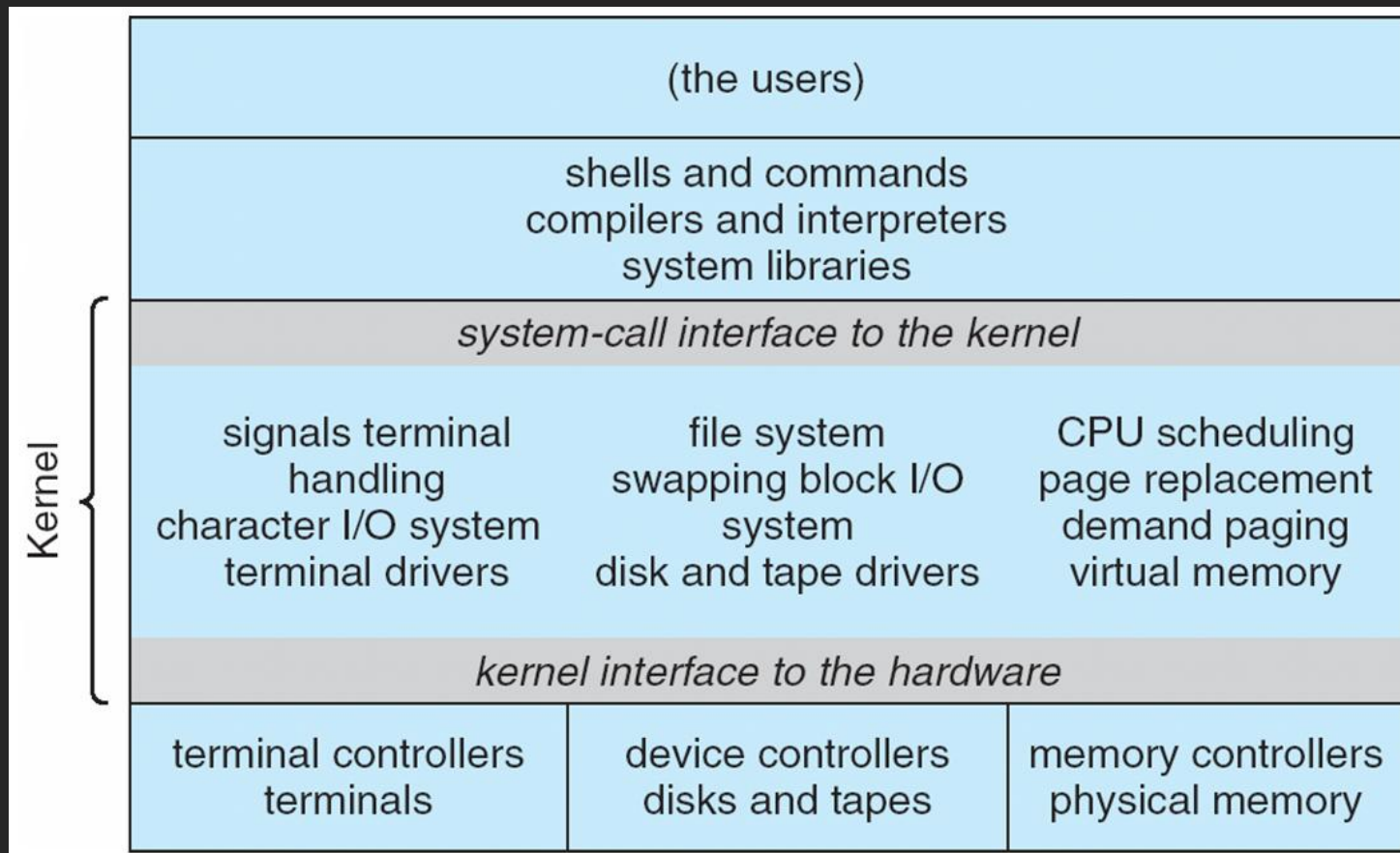
Estrutura do SO



Estrutura do SO

- UNIX tradicional: limitado pelo hardware da época
 - Estrutura limitada a dois níveis
 - Programas de sistema
 - Kernel: tudo abaixo da interface chamadas ao sistema e acima do hardware
 - Sistema de arquivos, escalonamento de CPU, gerenciamento de memória, etc.: um número muito grande de funções para um único nível

Estrutura do SO



Estrutura do SO

- Microkernel: move o máximo do kernel para espaço de usuário
 - Kernel: mínimo de ger. memória + processos + comunicação
 - Comunicação entre componentes: passagem de mensagens
 - Vantagens:
 - Mais fácil de desenvolver, estender e portar
 - Código mais confiável (menos código no modo kernel)
 - Falha em componente: impacto é limitado
 - Mais seguro
 - Desvantagens: baixo desempenho
 - Exemplos: Mach, Hurd

Estrutura do SO

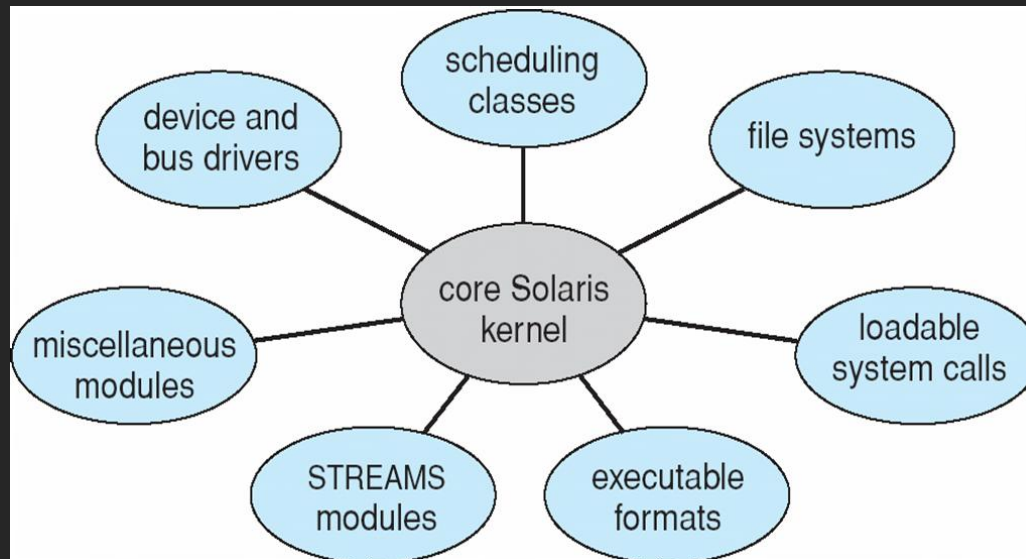
- Muitos SOs modernos implementam módulos de kernel
 - Kernel possui conjunto de componentes de núcleo (essenciais) e adiciona outros serviços via módulos
 - Módulos são carregados em tempo de inicialização ou execução
 - Também podem ser descarregados
 - Ex.: sistema de arquivos de mídia ejetável
 - Kernel pode oferecer novos serviços e correções sem ser recompilado
 - Exemplos serviços: drivers de dispositivos, sistemas de arquivos, protocolos de rede, criptografia, mapeamento de memória, correções do kernel

Estrutura do SO

- Módulos de kernel são similares a:
 - Abordagem em camadas: seções de kernel com interfaces bem definidas e protegidas
 - Porém, mais flexível: um módulo pode chamar qualquer outro
 - Microkernel: kernel apenas com funcionalidades essenciais e conhecimento de como carregar demais módulos
 - Porém, mais eficiente: sem troca de mensagens

Estrutura do SO

- Implementações modernas do UNIX, Solaris, Mac OS X, Linux e Windows usam módulos de kernel
 - Linux: lsmod, insmod, rmmod
- Módulos do Solaris:

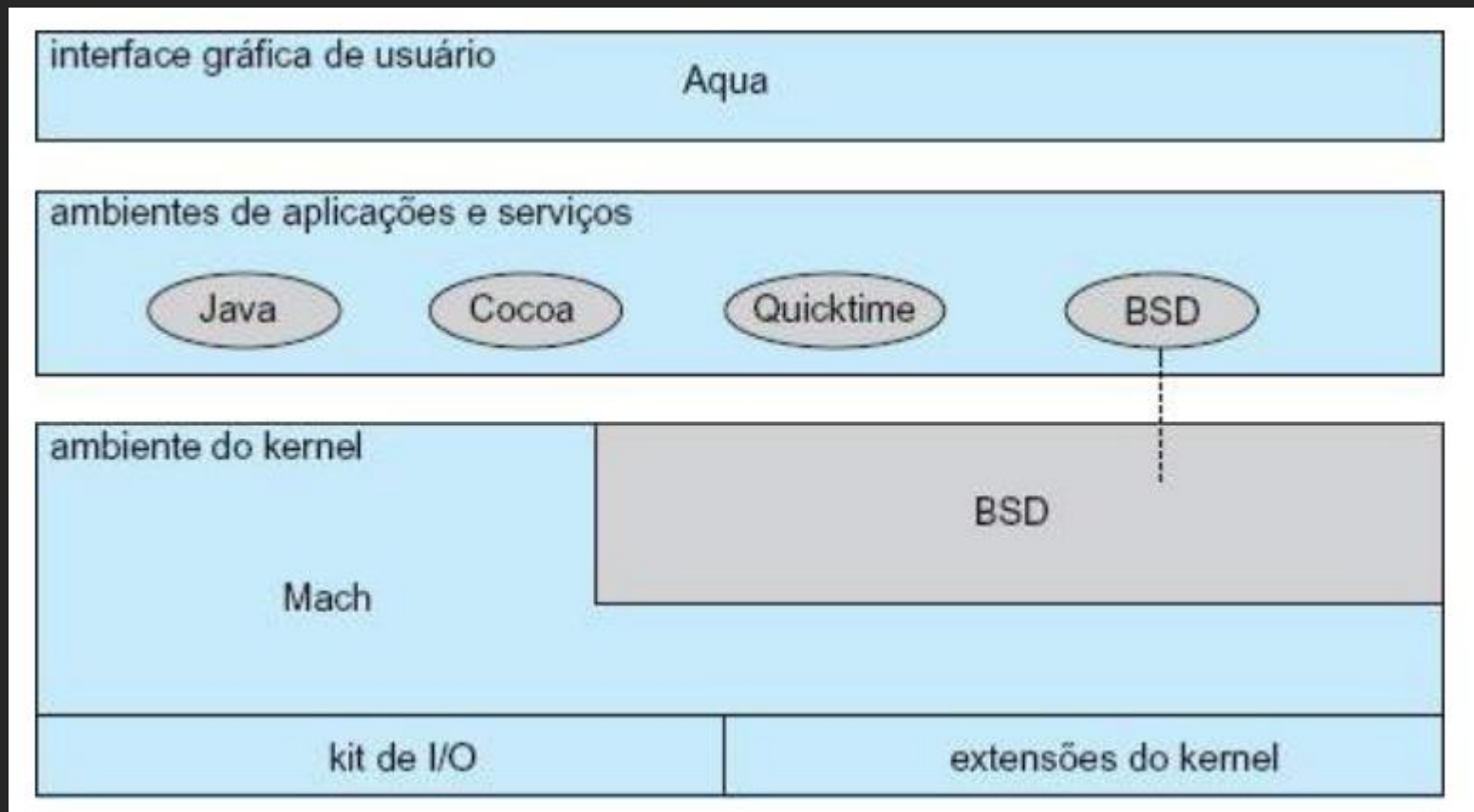


Estrutura do SO

- Mac OS X: abordagem híbrida
 - Em 3 camadas: interface gráfica, ambiente de aplicação/serviços, ambiente do kernel
 - Ambiente kernel: composto pelo microkernel Mach e kernel BSD UNIX
 - Mach: ger. memória, IPC, RPC, escalonamento de threads
 - BSD: interface de linha de comando BSD, rede, sistemas de arquivos, POSIX APIs
 - Suporta módulos de kernel (extensões do kernel)

Estrutura do SO

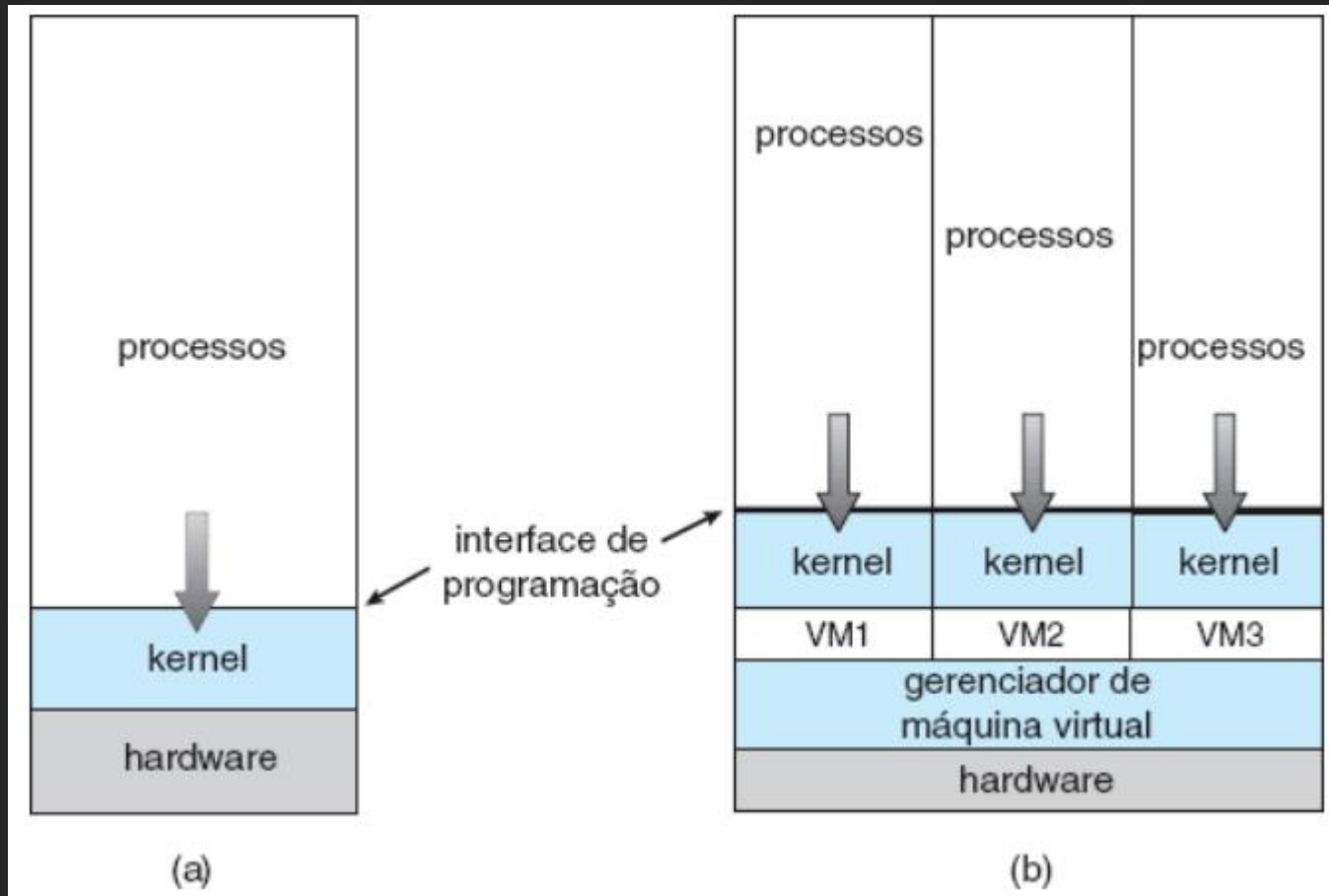
- Estrutura do Mac OS X:



Máquinas virtuais

- Máquina virtual (MV): abstrai hardware de um único computador em diversos ambientes de execução
 - Impressão de que cada ambiente possui computador privado
 - Componentes:
 - Hospedeiro: hardware que executa máquinas virtuais
 - Gerenciador de máquina virtual (VMM) ou hipervisor: cria e executa MV fornecendo interface igual ao hospedeiro
 - Convidado: processo (em geral um SO) que recebe cópia do hospedeiro

Máquinas virtuais



Sistemas sem MV (a) e com MV (b)

Máquinas virtuais

- Vantagens:

- Compartilhamento de hardware
- Uso de diferentes ambientes de execução
- Isolamento: protege sistema hospedeiro e outras MV de bugs/comprometimento em uma MV
- Desenvolvimento/pesquisa de SOs
- Consolidação: junção de dois sistemas em mesmo sistema

- Dificuldades:

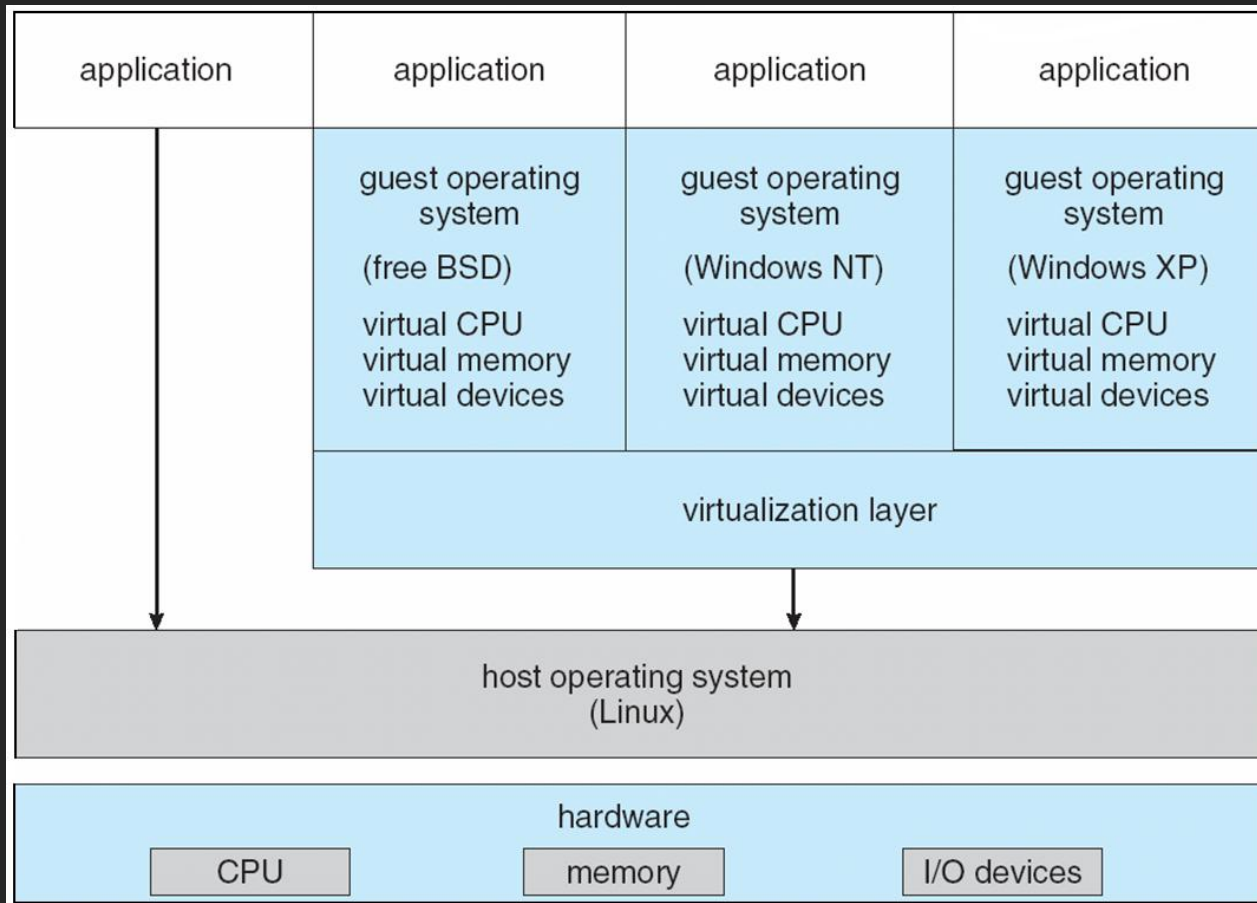
- Implementação, isolamento, desempenho

Máquinas virtuais

- Implementação: suporte especial de hardware
 - Máquina real: modo usuário e sistema
 - Máquina virtual: modo usuário virtual e sistema virtual
- Ex.: Chamada ao sistema feita por programa executando em modo usuário virtual
 - Tratamento feito pelo monitor da máquina virtual em execução na máquina real
 - Simulação da execução
 - Impacto: E/S pode ser mais rápida ou lenta que real
 - Muitos fatores a analisar: imprevisível

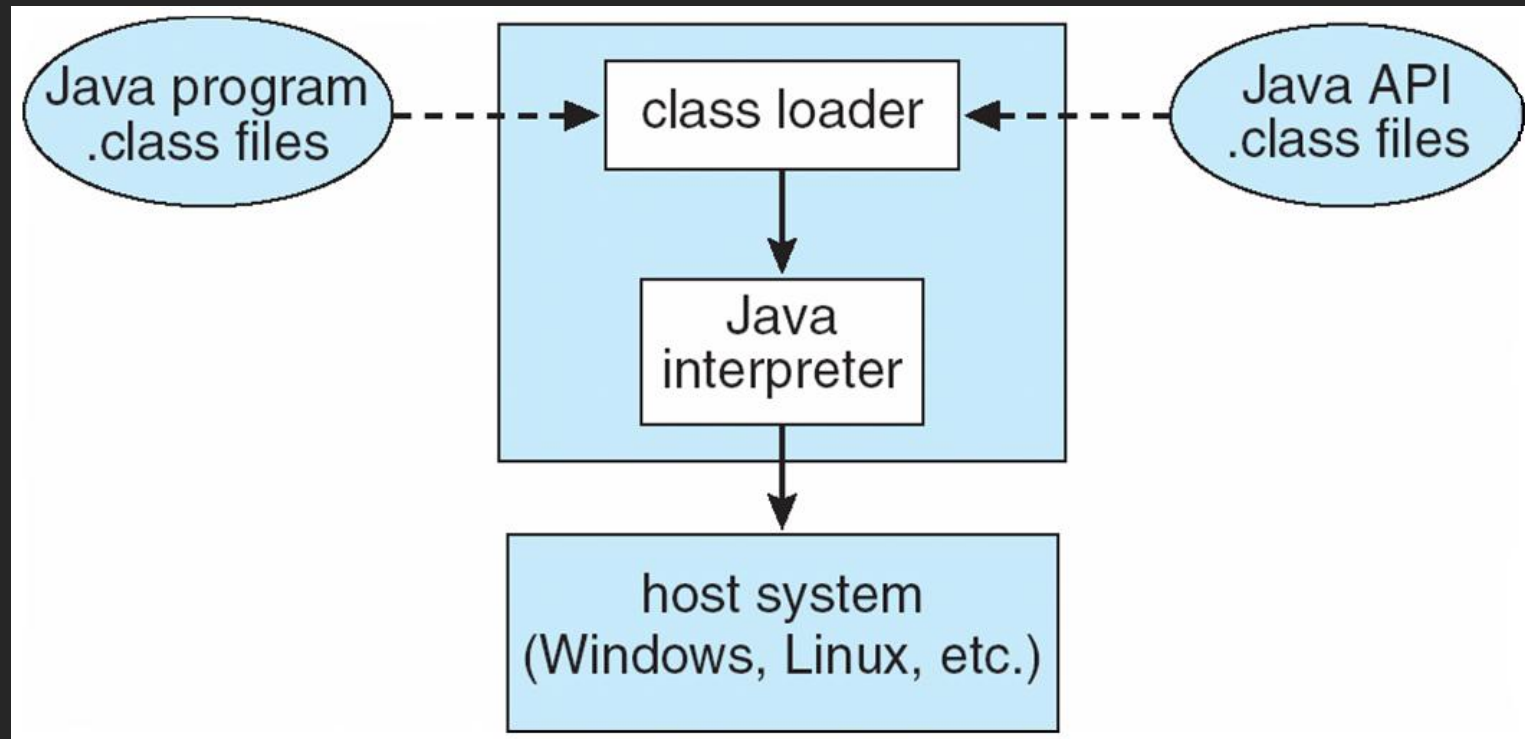
Máquinas virtuais

- Arquitetura da VMware:



Máquinas virtuais

- Máquina virtual Java:



Depuração e geração do SO

- Depuração: encontrar e consertar erros
 - Inclusive de desempenho: gargalos
 - Falha em aplicação: arquivo core dump
 - Memória do processo
 - Falha no SO: arquivo crash dump
 - Memória do kernel
 - Ferramentas auxiliares: ex.: dtrace

Depuração e geração do SO

- “Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.”
(Kernighan’s Law)

Depuração e geração do SO

- SOs são projetados para executar em qualquer máquina de uma determinada classe
 - Devem ser configurados para cada tal
 - Necessário obter informação de configuração do hardware do sistema
 - Quantas e quais CPUs, como formatar disco, quantidade de memória (acesso ilegal à memória), dispositivos disponíveis e seus tipos, quais opções do SO a serem instaladas